Data Info Manual

1. User Interface (UI)

In this section of the user interface, your experimental data will be displayed. Depending on your experiment, protein, phosphosite, and/or phosphoprotein data may appear. All datasets can be downloaded directly from the interface.

The first section lists the names of the raw files used for the analysis:

Protein, Phospho & Phosphoprotein Data Upload

Protein file: C:\PythonProject\ProteomisCoPYlot\data/ServerView/main/20241210_154937_20241018_JM_WP_8cells_ReportPG.tsv

Phospho file: C:\PythonProject\ProteomisCoPYlot\data/ServerView/main/20250325_141041_20241018_JM_PP_8cells_Report_pp_collapsed.tsv

Phosphoprotein file: C:\PythonProject\ProteomisCoPYlot\data/ServerView/main/20250325_141041_20241018_JM_PP_8cells_Report_phosprot.csv

Figure 1: Raw file names in the UI.

The second section displays the $\log_2$-transformed datasets. These data can also be filtered and/or imputed. The corresponding column headers indicate the dataset type:

Protein Data (Log2 & Filtered)

	PG.ProteinAccessions	PG.ProteinDescriptions	ProteinNames
31	A0A087WW06;Q96AY4	Tetratricopeptide repeat domain 28;Tetratricopeptide repeat protein 28	A0A087WW06_HUMAN;TTC28_HUMAN
32	A0A087WW88;Q9UQN3	Charged multivesicular body protein 2B;Charged multivesicular body protein 2b	A0A087WW88_HUMAN;CHM2B_HUMAN
33	A0A087WWD4;P13591	Neural cell adhesion molecule 1	A0A087WWD4_HUMAN;NCAM1_HUMAN
34	A0A087WWF6;A0A8V8TMJ5;H7C1B3;P49005	DNA polymerase delta subunit 2	A0A087WWF6_HUMAN;A0A8V8TMJ5_HUMAN;H
35	A0A087WWH3;Q9NZM4	BRD4-interacting chromatin-remodeling complex-associated protein	A0A087WWH3_HUMAN;BICRA_HUMAN
36	A0A087WWM0;O43617	Trafficking protein particle complex subunit;Trafficking protein particle complex sub	A0A087WWM0_HUMAN;TPPC3_HUMAN
37	A0A087WWP4;A0A499FJY3;Q96T37	RNA binding motif protein 15;RNA binding motif protein 15;RNA-binding protein 15	A0A087WWP4_HUMAN;A0A499FJY3_HUMAN;RB
38	A0A087WWP8;Q5VWQ0	Round spermatid basic protein 1;Lysine-specific demethylase 9	A0A087WWP8_HUMAN;RSBN1_HUMAN
39	A0A087WWT1;P21912	Succinate dehydrogenase [ubiquinone] iron-sulfur subunit, mitochondrial;Succinate	A0A087WWT1_HUMAN;SDHB_HUMAN
40	A0A087WWU8	Tropomyosin 3	A0A087WWU8_HUMAN
41	A0A087WX23	Paternally expressed 10	A0A087WX23_HUMAN

Phospho Data (Log2 & Filtered)

	20241018_nElute_tims1_JM_PP_8cell_C_0min_1_Slot1-01_1_8845	20241018_nElute_tims1_JM_PP_8cell_C_0min_2_Slot1-01_1_8870	20241018_nElute_tims1_JM_PP_8cell_C_0min_3_Slot1-
54	1.4551	3.9565	
55	0.7596	None	
56	2.5049	5.7407	
57	7.8402	6.9261	
58	6.5896	6.1156	
59	6.7958	6.1234	
60	8.1095	7.9371	

Figure 2: Displayed data sets.

At the bottom of the interface, you can review the processing steps applied to each dataset. This includes any filtering and/or imputation that was performed. The resulting dimensions of each dataset, reported as rows and columns, are also shown. If a phosphoprotein dataset was generated, the method used to collapse phosphosites is displayed as well.

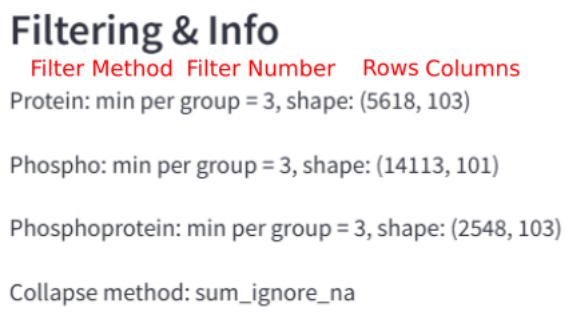

Figure 3: Data filtering and information.

2. Filtering Methods

Filtering can be performed using two methods: "per group" and "in at least one group." Each filtering method is associated with a filtering value, which is a non-negative integer. This value specifies the minimum number of valid (non-NA) entries required in a row for the row to be retained according to the selected method.

2.1 Filtering "per group"

The "per group" filtering method requires that **each group or condition** contains at least the specified number of valid values for a row to be retained.

Table 1: Filtering "per group"

Feature	A1	A2	A3	B1	B2	B3	Filter num
1	1.34	2.45	1.42	NA	3.43	3.44	3
2	1.34	2.45	1.42	5.32	3.43	3.44	3

In this example, there are six samples distributed across two conditions (Condition A and Condition B). For Feature 1, at least three valid values are required in each condition. This requirement is met for Condition A but not for Condition B, which contains only two valid values. As a result, Feature 1 is discarded, and its row is removed. For Feature 2, both conditions meet the minimum requirement, so Feature 2 is retained in the filtered dataset.

2.2 Filtering "in at least one group"

The "in at least one group" filtering method requires that a row contains at least the specified number of valid values in **at least one condition** for it to be retained.

Table 2: Filtering "in at least one group"

Feature	A1	A2	A3	B1	B2	B3	Filter num
1	1.34	2.45	1.42	NA	3.43	3.44	3
2	1.34	NA	1.42	NA	3.43	3.44	3

In this example, there are six samples distributed across two conditions (Condition A and Condition B). For Feature 1, at least three valid values are required in at least one condition. This requirement is satisfied by Condition A, which contains three valid values, so Feature 1 is retained in the filtered dataset. For Feature 2, neither Condition A nor Condition B contains three valid values (each has only two valid values). As a result, Feature 2 is discarded, and its row is removed from the filtered dataset.

3. Collapse Method

The **"transform_phosprot"** function provides four methods to summarize phosphosite-level data into phosphoprotein-level data. Each method handles missing values differently, allowing flexibility depending on the nature of the experimental data and the desired treatment of incomplete measurements.

The first method, "**sum_ignore_na"**, also sums values across samples, but it ignores any missing values. Only the present, non-missing values contribute to the sum. This method is appropriate when one wishes to retain phosphoproteins that have partial measurements, without letting missing values prevent aggregation.

Table 3: Collapse "transform_phosprot" using "sum_ignore_na"

Feature	A1	A2	A3	B1	B2	B3
ABC_Y211	1	NA	3	NA	4	6
ABC_S344	2	NA	4	3	NA	5
ABC	3	NA	7	3	4	11

The second method, "**sum_propagate_na"**, calculates the sum of values across samples but propagates missing values. In this method, if any value is missing for a phosphoprotein in a given condition, the aggregated sum becomes NA. This ensures that uncertainty due to missing data is carried through to the phosphoprotein level and can help avoid misleading totals when data is incomplete.

Table 4: Collapse "transform_phosprot" using "sum_propagate_na"

Feature	A1	A2	A3	B1	B2	B3
ABC_Y211	1	NA	3	NA	4	6
ABC_S344	2	NA	4	3	NA	5
ABC	3	NA	7	NA	NA	11

The third method, "**sum_mean_impute"**, begins by imputing missing values with the mean of the corresponding condition. Once missing values are filled, the function sums the values across all samples for each phosphoprotein. This approach is useful when one wants to retain all phosphoproteins, assuming that missing values can be reasonably approximated by the condition-specific mean.

Table 5: Collapse “transform_phosprot” using “sum_mean_impute”

Feature	A1	A2	A3	B1	B2	B3
ABC_Y211	1	NA	3	NA	4	6
ABC_S344	2	NA	4	3	NA	5
ABC	3	5	7	8	8	11

The fourth method, “**mean”**, calculates the mean of the values across all samples for each phosphoprotein. Missing values are ignored in the calculation, so the mean is determined only from available measurements. This approach is suitable when relative abundance is more informative than total abundance, and it reduces the impact of missing measurements without discarding the protein entirely.

Table 6: Collapse “transform_phosprot” using “sum_mean”

Feature	A1	A2	A3	B1	B2	B3
ABC_Y211	1	NA	3	NA	4	6
ABC_S344	2	NA	4	3	NA	5
ABC	1.5	NA	3.5	3	4	5.5